Freestanding Library: Easy [utilities], [ranges], and [iterators]

Document number: P1642R3
Date: 2020-05-24
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: Library Evolution Working Group

Change history

R3

- Rebased to N4861
- Putting wording back (in the form of instructions to editor)
- Per-header feature test macro, similar to `constexpr` macros

R2

- Rebased to N4842
- No longer including `istream_view`

R1

- Adding section summarizing what is getting added.
- Adding all of [ranges] and most of [iterators]
- Adding `uses_allocator_construction_args`
- Adding feature test macro
- Now discussing split overload sets.
- Dropping wording for now

R0

Branching from P0829R4. This "omnibus" paper is still the direction I am aiming for. However, it is too difficult to review. It needs to change with almost every meeting. Therefore, it is getting split up into smaller, more manageable chunks.

Limiting paper to the [utilities], [ranges], and [iterators] clauses.

Introduction

This paper proposes adding many of the facilities in the [utilities], [ranges], and [iterators] clause to the freestanding subset of C++. The paper will only be adding complete entities, and will not tackle partial classes. In other words, classes like `pair` and `tuple` are being added, but trickier classes like `optional`, `variant`, and `bitset` will come in another paper.

The `<memory>` header has a dependency on facilities in `<ranges>` and `<iterator>`, so those headers (and clauses) are addressed as well.

Motivation and Design

Many existing facilities in the C++ standard library could be used without trouble in freestanding environments. This series of papers will specify the maximal subset of the C++ standard library that does not require an OS or space overhead.

For a more in depth rationale, see P0829.

`<optional>`, `<variant>`, and `<bitset>` are not in this paper, as all have non-essential functions that can throw an exception. `<charconv>` is not in this paper as it will require us to deal with the thorny issue of overload sets involving floating point and non-floating point types. I plan on addressing all four of these headers in later papers, so that the topics in question can be debated in relative isolation.

Splitting overload sets

Modifying an overload set needs to be treated with great care. Ideally, libraries built in freestanding environments will have the same semantics (including selected overloads) when built in a hosted environment. I don't think that goal is 100% achievable, as sufficiently clever programmers will be able to detect the difference and modify behavior accordingly.

My approach will be to avoid splitting overload sets that could cause accidental freestanding / hosted differences. In future papers, I may need to lean on `= delete` techniques to avoid silent behavior changes, but this paper hasn't needed that approach.

`swap` has `shared_ptr`, `weak_ptr`, and `unique_ptr` overloads, but this paper only marks the `unique_ptr` overload as freestanding. `<memory>` has many algorithms with `ExecutionPolicy` overloads. This paper does not mark the `ExectuionPolicy` overloads as freestanding. I was unable to come up with any compelling "accidental" way to select one `swap` or algorithm overload in freestanding, but a different one in hosted.

Also note that the `swap` overload set visible to a given translation unit is already indeterminate. A user may include `<memory>` which guarantees the smart pointer overloads, but an implementation could expose any (or none!) of the other `swap` overloads from other STL headers.

Justification for omissions

The following functions and classes rely on dynamic memory allocation and exceptions:

- `make_obj_using_allocator`
- `make_unique`
- `make_unique_default_init`
- `shared_ptr`
- `weak_ptr`
- `function`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`

The following classes rely on iostreams facilities. iostreams facilities use dynamic memory allocations and rely on the operating system.

- `istream_iterator`
- `ostream_iterator`
- `istreambuf_iterator`
- `ostreambuf_iterator`
- `basic_istream_view`
- `istream_view`

The ExecutionPolicy overloads of algorithms are of minimal utility on systems that do not support C++ threads.

Feature test macros

Add the following feature test macros to **freestanding only**:

Name	Header
`__cpp_lib_freestanding_utility`	`<utility>`
`__cpp_lib_freestanding_tuple`	`<tuple>`

Name	Header
`__cpp_lib_freestanding_ratio`	`<ratio>`
`__cpp_lib_freestanding_memory`	`<memory>`
`__cpp_lib_freestanding_functional`	`<functional>`
`__cpp_lib_freestanding_iterator`	`<iterator>`
`__cpp_lib_freestanding_ranges`	`<ranges>`

The following, existing feature test macros cover some features that I am making freestanding, and some features that I am not requiring to be freestanding. These feature test macros won't be required in freestanding, as they could cause substantial confusion when the hosted parts of those features aren't available. The header-based `__cpp_lib_freestanding_*` macros should provide a suitable replacement in freestanding environments.

- `__cpp_lib_boyer_moore_searcher`
- `__cpp_lib_constexpr_dynamic_alloc`
- `__cpp_lib_ranges`
- `__cpp_lib_raw_memory_algorithms`

Wording

Wording assumes that P1641 has been applied.

Full headers

Instructions to the editor:
Please add a *// freestanding* comment to the beginning of the following synopses. These headers are entirely freestanding.

- [utility.syn]
- [tuple.syn]
- [ratio.syn]

Change in [memory.syn]

Instructions to the editor:
Please add a *// freestanding* comment to the following declarations:

- `pointer_traits`
- `to_address`
- `align`
- `assume_aligned`
- `allocator_arg_t`
- `allocator_arg`
- `uses_allocator`
- `uses_allocator_v`
- `uses_allocator_construction_args`
- `allocator_traits`
- [specialized.algorithms], except for the `ExecutionPolicy` overloads. This includes the algorithms in the `ranges` namespace
- `default_delete`
- `unique_ptr`
- `unique_ptr` overload of `swap`
- relational operators (including three-way / spaceship) involving `unique_ptr`
- `hash`
- `unique_ptr` specialization of `hash`
- `atomic`

Note: the following portions of `<memory>` are **omitted**. No change to the working draft should be made for these declarations:

- [util.dynamic.safety], pointer safety
- `make_obj_using_allocator`
- `uninitialized_construct_using_allocator`
- `allocator` and associated comparisons
- `ExecutionPolicy` overloads in [specialized.algorithms]
- `make_unique`
- `make_unique_default_init`
- All `operator<<` overloads
- `bad_weak_ptr`
- `shared_ptr`
- `make_shared`
- `allocate_shared`
- `make_shared_default_init`
- `allocate_shared_default_init`
- relational operators (including three-way / spaceship) involving `shared_ptr`
- `shared_ptr` overload of `swap`
- `static_pointer_cast`
- `dynamic_pointer_cast`
- `const_pointer_cast`
- `reinterpret_pointer_cast`
- `get_deleter`
- `weak_ptr`
- `weak_ptr` overload of `swap`
- `owner_less`
- `enable_shared_from_this`
- `shared_ptr` specialization of `hash`
- `shared_ptr` specialization of `atomic`
- `weak_ptr` specialization of `atomic`

Change in [functional.syn]

Instructions to the editor:
Please add a *// freestanding* comment to every entity in `<functional>` **except** for the following entities:

- `bad_function_call`
- `function`
- `function` overloads of `swap`
- `function` overloads of `operator==`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`

Change in [iterator.synopsis]

Instructions to the editor:
Please add a *// freestanding* comment to every entity in `<iterator>` **except** for the following entities:

- `istream_iterator` and associated comparison operators
- `ostream_iterator`
- `istreambuf_iterator` and associated comparison operators
- `ostreambuf_iterator`

Change in [ranges.syn]

Instructions to the editor:
Please add a *// freestanding* comment to every entity in `<ranges>` **except** for the following entities:

- `basic_istream_view`
- `istream_view`

Change in [version.syn]

Please add a *// freestanding* comment to each of the following macro definitions:

- `__cpp_lib_addressof_constexpr`
- `__cpp_lib_allocator_traits_is_always_equal`
- `__cpp_lib_apply`
- `__cpp_lib_as_const`
- `__cpp_lib_assume_aligned`
- `__cpp_lib_atomic_value_initialization`
- `__cpp_lib_bind_front`
- `__cpp_lib_constexpr_functional`
- `__cpp_lib_constexpr_iterator`
- `__cpp_lib_constexpr_memory`
- `__cpp_lib_constexpr_tuple`
- `__cpp_lib_constexpr_utility`
- `__cpp_lib_exchange_function`
- `__cpp_lib_integer_sequence`
- `__cpp_lib_invoke`
- `__cpp_lib_make_from_tuple`
- `__cpp_lib_make_reverse_iterator`
- `__cpp_lib_nonmember_container_access`
- `__cpp_lib_not_fn`
- `__cpp_lib_null_iterators`
- `__cpp_lib_ssize`
- `__cpp_lib_to_address`
- `__cpp_lib_transparent_operators`
- `__cpp_lib_tuple_element_t`
- `__cpp_lib_tuples_by_type`
- `__cpp_lib_unwrap_ref`

Please add the following macro definitions:

```
#define __cpp_lib_freestanding_functional   202005L // also in <functional>, freestanding only
#define __cpp_lib_freestanding_iterator     202005L // also in <iterator>, freestanding only
#define __cpp_lib_freestanding_memory       202005L // also in <memory>, freestanding only
#define __cpp_lib_freestanding_ranges       202005L // also in <ranges>, freestanding only
#define __cpp_lib_freestanding_ratio        202005L // also in <ratio>, freestanding only
#define __cpp_lib_freestanding_tuple        202005L // also in <tuple>, freestanding only
#define __cpp_lib_freestanding_utility      202005L // also in <utility>, freestanding only
```

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing P0829.

Thanks to Odin Holmes for providing feedback and helping publicize P0829.

Thanks to Paul Bendixen for providing feedback while prototyping P0829.

Similar work was done in the C++11 timeframe by Lawrence Crowl and Alberto Ganesh Barbati in N3256.